

Relatorio Tecnico - Maquinas Abstratas

Relatório Técnico — Implementação de Máquinas Abstratas

Disciplina: Fundamentos Teóricos da Computação **Instituição:** Faculdade Cotemig **Professor:** Júlio César da Silva
Equipe: Anna Julya Fernandes (72401192) *(demais integrantes a preencher)* **Data:** junho de 2026

Como gerar o PDF exigido (docs/relatorio.pdf): copie este conteúdo para o Word ou Google Docs, aplique a formatação ABNT (Times New Roman ou Arial, tamanho 12, espaçamento 1,5; margens superior/esquerda 3 cm e inferior/direita 2 cm) e exporte como PDF. Alternativamente, use `pandoc relatorio.md -o relatorio.pdf`.

1. Introdução

Este trabalho implementa, na linguagem C# (.NET 6), três máquinas abstratas fundamentais da Teoria da Computação, organizadas em ordem crescente de poder computacional: o Autômato Finito Determinístico (AFD), o Autômato de Pilha (AP) com reconhecimento por pilha vazia e a Máquina de Turing (MT). O objetivo é traduzir fielmente as definições matemáticas formais em estruturas de dados e algoritmos, evidenciando, na prática, a hierarquia de Chomsky: linguagens regulares, livres de contexto e recursivamente enumeráveis. Cada máquina foi construída para reconhecer (ou, no caso da MT, também computar) uma linguagem específica, com exibição passo a passo das configurações para fins de verificação e depuração.

2. Fundamentação Teórica

2.1 Autômato Finito Determinístico (AFD)

Um AFD é uma 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$ onde Q é o conjunto finito de estados, Σ o alfabeto de entrada, $\delta: Q \times \Sigma \rightarrow Q$ a função de transição (total e determinística), $q_0 \in Q$ o estado inicial e $F \subseteq Q$ o conjunto de estados de aceitação. Uma cadeia w é aceita se a computação a partir de q_0 , consumindo w , termina em um estado de F . AFDs reconhecem exatamente as **linguagens regulares**.

Linguagem-alvo: $L1 = \{ w \in \{a,b\}^* \mid w \text{ termina com "ab"} \}$.

AFD construído: $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, q_0 inicial, $F = \{q_2\}$.

δ	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$* q_2$	q_1	q_0

Semântica dos estados: **q0** = ainda não há sufixo **ab** em formação; **q1** = o último símbolo lido foi **a** (candidato a iniciar **ab**); **q2** = os dois últimos símbolos lidos foram **ab** (aceitação).

Cálculo manual (2 cadeias):

- bab**: $q_0 \xrightarrow{b} q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2$. Termina em $q_2 \in F \rightarrow$ **ACEITA**.
- ba**: $q_0 \xrightarrow{b} q_0 \xrightarrow{a} q_1$. Termina em $q_1 \notin F \rightarrow$ **REJEITA**.

2.2 Autômato de Pilha com aceitação por pilha vazia

Um AP é uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \varnothing)$ onde Γ é o alfabeto da pilha, $Z_0 \in \Gamma$ o símbolo inicial da pilha e $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$ a função de transição (possivelmente não-determinística). Na **aceitação por pilha vazia**, o conjunto de estados finais é vazio: w é aceita se existe uma computação que consome toda a entrada e esvazia a pilha. APs reconhecem as **linguagens livres de contexto**.

Linguagem-alvo: $L_2 = \{ a^n b^n \mid n \geq 1 \}$. AP determinístico que empilha um A para cada a e desempilha um A para cada b ; um λ -movimento remove Z_0 ao final.

Cálculo manual (2 cadeias):

- $ab : (q_0, ab, Z) \vdash (q_0, b, AZ) \vdash (q_1, \epsilon, Z) \vdash (q_1, \epsilon, \epsilon)$. Pilha vazia e entrada consumida $\rightarrow$ **ACEITA**.
- $aab : (q_0, aab, Z) \vdash (q_0, ab, AZ) \vdash (q_0, b, AAZ) \vdash (q_1, \epsilon, AZ)$. Entrada consumida, mas a pilha contém $AZ \neq$ vazia $\rightarrow$ **REJEITA**.

Desafio L3 (palíndromos): não-determinístico — empilha a primeira metade, “adivinha” o meio (λ -movimento para comprimento par; consumo do símbolo central para comprimento ímpar) e desempilha casando a segunda metade.

2.3 Máquina de Turing (MT)

Uma MT é uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ com $\Gamma \supseteq \Sigma$ contendo o símbolo branco $\sqcup \notin \Sigma$, e $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$. A fita é infinita, o cabeçote lê/escreve e move-se para a esquerda (L) ou direita (R). MTs reconhecem as **linguagens recursivamente enumeráveis** e formalizam a noção de algoritmo.

Linguagem-alvo: $L_4 = \{ a^n b^n c^n \mid n \geq 1 \}$, linguagem **sensível ao contexto** (não livre de contexto). Estratégia de marcação: a cada passagem, marca um $a \rightarrow X$, um $b \rightarrow Y$, um $c \rightarrow Z$, e retorna ao início; ao fim verifica que sobraram apenas Y s e Z s na ordem correta.

Cálculo manual (2 cadeias):

- $abc : \text{marca } a \rightarrow X, b \rightarrow Y, c \rightarrow Z; \text{ volta; em } q_0 \text{ lê } Y \text{ (sem mais } a \text{), verifica } YZ \sqcup \rightarrow \text{ACEITA.}$
- $aabbc : \text{marca um par (a,b,c) e, na segunda passagem, marca o segundo } a \text{ e o segundo } b, \text{ mas não há segundo } c; \text{ o cabeçote alcança o branco em } q_2 \text{ sem transição definida } \rightarrow \text{REJEITA.}$

Desafio $f(n) = n+1$ (unário): a MT percorre o bloco de 1 s até o primeiro branco e o substitui por 1 , acrescentando exatamente um símbolo.

3. Descrição da Implementação

Todas as máquinas foram implementadas sem bibliotecas externas de autômatos, usando apenas coleções nativas do C#, e cada estrutura de dados foi nomeada de forma a refletir diretamente a tupla formal.

- AFD (Afd.cs):** os cinco componentes da 5-tupla são propriedades explícitas (Estados, Alfabeto, Transicoes, EstadoInicial, EstadosAceitacao). A função δ é um `Dictionary<(string, char), string>`. O método `Simular` percorre a cadeia símbolo a símbolo, registrando o rastro; o desafio de carga dinâmica usa `System.Text.Json` para ler `afd.json`.
- AP (AutomatoPilha.cs):** a pilha é uma `Stack<char>`, conforme exigido, e os λ -movimentos são representados por `'\0'`. Como L_3 é não-determinística, a simulação faz uma **busca em profundidade** sobre as configurações instantâneas, clonando a pilha a cada ramo e usando um conjunto de configurações visitadas (mais um limite de passos) para evitar laços infinitos de λ -movimentos. A aceitação é verificada exclusivamente por pilha vazia + entrada consumida — nenhum estado final é usado.
- MT (MaquinaTuring.cs):** a fita é um `Dictionary<int, char>` dinâmico (`branco = '_'`), e δ é um

Dictionary<(string,char),(string,char,char)>. A execução exibe a cada passo o estado, a fita com [] ao redor do símbolo sob o cabeçote e a posição do cabeçote; há qaccept / qreject explícitos, contador de passos e limite configurável. Transições não definidas levam à rejeição automática.

4. Resultados e Testes

4.1 Parte 1 — AFD (L1)

Cadeia	Esperado (enunciado)	Obtido	Observação
ab	ACEITA	ACEITA	caso mínimo
aab	REJEITA*	ACEITA	*o enunciado está incorreto: aab termina em ab
bab	ACEITA	ACEITA	prefixo irrelevante
ababab	ACEITA	ACEITA	múltiplas ocorrências
ba	REJEITA	REJEITA	termina com a
ϵ	REJEITA	REJEITA	sem caracteres
b	REJEITA	REJEITA	um símbolo

4.2 Parte 2 — AP ($L2 = a^n b^n$)

Cadeia	Esperado	Obtido
ab	ACEITA	ACEITA
aabb	ACEITA	ACEITA
aaabbb	ACEITA	ACEITA
aab	REJEITA	REJEITA
abb	REJEITA	REJEITA
ba	REJEITA	REJEITA
ϵ	REJEITA	REJEITA
abab	REJEITA	REJEITA

4.3 Parte 2 — AP ($L3 = \text{palíndromos}$)

Cadeia	Esperado	Obtido
a	ACEITA	ACEITA
aba	ACEITA	ACEITA
abba	ACEITA	ACEITA
ab	REJEITA	REJEITA
aab	REJEITA	REJEITA

4.4 Parte 3 — MT ($L4 = a^n b^n c^n$)

Cadeia	Esperado	Obtido
abc	ACEITA	ACEITA

Cadeia	Esperado	Obtido
aabbcc	ACEITA	ACEITA
aaabbbccc	ACEITA	ACEITA
aabbc	REJEITA	REJEITA
ab	REJEITA	REJEITA
abcabc	REJEITA	REJEITA
ϵ	REJEITA	REJEITA

4.5 Parte 3 — MT computadora $f(n) = n+1$

Fita de entrada	Saída esperada	Saída obtida
1	11	11
111	1111	1111
11111	111111	111111

5. Análise Comparativa

As três máquinas formam uma hierarquia estrita de poder computacional. O **AFD** tem memória finita (apenas o estado corrente) e reconhece somente linguagens regulares; é incapaz de “contar” arbitrariamente, por isso não reconhece $a^n b^n$. O **AP** acrescenta uma pilha (memória ilimitada, mas com acesso restrito ao topo, LIFO), o que permite reconhecer linguagens livres de contexto como $a^n b^n$ e palíndromos; ainda assim, uma única pilha não basta para $a^n b^n c^n$, pois ao verificar os b a contagem dos a é destruída. A **MT**, com fita infinita de leitura e escrita bidirecional, supera ambos: reconhece $a^n b^n c^n$ e, mais do que reconhecer, **computa funções** (como o incremento unário). Em resumo: Regulares $\subsetneq$ Livres de Contexto $\subsetneq$ Sensíveis ao Contexto $\subseteq$ Recursivamente Enumeráveis, com AFD, AP e MT como reconhecedores canônicos das três primeiras (a MT cobrindo as duas últimas).

6. Conclusão

A implementação evidenciou, de forma concreta, por que cada modelo possui o poder que a teoria lhe atribui: a diferença entre eles está essencialmente na **memória disponível** e em **como ela pode ser acessada**. Traduzir as tuplas formais diretamente em estruturas de dados (dicionários para δ , `Stack<char>` para a pilha, `Dictionary<int, char>` para a fita) tornou a correspondência entre teoria e código transparente e facilitou a depuração via configurações instantâneas. O exercício reforçou a compreensão da hierarquia de Chomsky e da Tese de Church-Turing.

7. Referências

- SIPSER, Michael. *Introduction to the Theory of Computation*. 3. ed. Boston: Cengage, 2013.
- HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. *Introdução à Teoria de Autômatos, Linguagens e Computação*. 2. ed. Rio de Janeiro: Campus/Elsevier, 2003.
- MENEZES, Paulo Blauth. *Linguagens Formais e Autômatos*. 6. ed. Porto Alegre: Bookman, 2010.

Apêndice — Questões Reflexivas

Parte 1 — AFD

1. Por que L_1 é regular? Expressão regular equivalente. L_1 é regular porque pode ser descrita por uma expressão regular e reconhecida por um AFD com número finito de estados — exatamente a caracterização das linguagens regulares (Teorema de Kleene). Para reconhecer “termina com ab ”, basta lembrar, a qualquer momento, apenas se o sufixo lido até agora é vazio, é a ou é ab ; isso são três situações finitas, sem necessidade de contagem. A expressão regular equivalente é $(a|b)^*ab$: qualquer prefixo sobre $\{a,b\}$ seguido obrigatoriamente do bloco ab no final. Como a quantidade de informação a memorizar é limitada (constante), um autômato de memória finita é suficiente, o que confirma a regularidade.

2. 5-tupla formal e justificativa de cada estado. $M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$. q_0 representa “nenhum progresso rumo ao sufixo ab ” — todo b mantém em q_0 e todo a inicia um possível sufixo. q_1 representa “acabei de ler um a ”: dele, um b completa ab (vai a q_2) e outro a mantém o candidato (permanece em q_1). q_2 representa “os dois últimos símbolos foram ab ” e é o único estado de aceitação; um a seguinte recomeça um candidato (q_1) e um b destrói o sufixo (volta a q_0). A escolha de três estados é mínima: precisamos distinguir exatamente esses três sufixos possíveis (ϵ , a , ab).

3. E se δ não fosse total? Tratamento de símbolos inválidos. Formalmente, um AFD exige δ total; quando ela é parcial, o comportamento convencional é assumir um “estado morto” (poço) implícito do qual não se sai e que não aceita — equivalente a rejeitar a cadeia. Na implementação, δ é um dicionário que pode não conter uma chave: quando $(estado, símbolo)$ não existe, o método `Simular` rejeita a cadeia imediatamente. Para símbolos **fora do alfabeto** Σ (por exemplo c), o programa também rejeita e emite uma mensagem explicando que o símbolo não pertence a Σ . Assim, entradas inválidas não quebram a execução: são tratadas como rejeição segura, preservando o determinismo.

Parte 2 — Autômato de Pilha

1. Por que $L_2 = a^n b^n$ não é regular? (Lema do Bombeamento). Suponha, por absurdo, que L_2 seja regular. Pelo Lema do Bombeamento, existe um comprimento de bombeamento p tal que toda cadeia $w \in L_2$ com $|w| \geq p$ pode ser escrita como $w = xyz$, com $|xy| \leq p$, $|y| \geq 1$, e $xy^i z \in L_2$ para todo $i \geq 0$. Tome $w = a^p b^p \in L_2$. Como $|xy| \leq p$, o trecho xy está inteiramente na região de a s, logo $y = a^k$ com $k \geq 1$. Bombeando $i = 2$, obtemos $xy^2 z = a^{p+k} b^p$, que tem mais a s do que b s e portanto **não pertence a L_2** . Isso contradiz o lema; logo L_2 não é regular. Intuitivamente, um AFD teria de “contar” os a s para compará-los aos b s, o que exige memória ilimitada, indisponível em um autômato de estados finitos.

2. Aceitação por pilha vazia $\times$ por estado final são equivalentes? Sim, são equivalentes em poder de reconhecimento: a classe de linguagens aceitas por APs por pilha vazia é exatamente a classe aceita por APs por estado final — ambas são as linguagens livres de contexto. A demonstração é construtiva. Dado um AP que aceita por estado final, constrói-se outro que aceita por pilha vazia adicionando um novo símbolo de fundo x_0 (abaixo de Z_0) e, dos estados finais, λ -movimentos que esvaziam a pilha; um estado “dreno” garante que apenas as cadeias antes aceitas esvaziam a pilha. Reciprocamente, dado um AP que aceita por pilha vazia, adiciona-se x_0 no fundo e um único estado final alcançado por λ -movimento quando x_0 aparece no topo (indicando pilha logicamente vazia). As construções preservam a linguagem, provando a equivalência.

3. Papel do símbolo Z_0 . Z_0 é o marcador de **fundo da pilha**, empilhado antes do início da computação. Ele cumpre dois papéis na implementação: (i) permite detectar quando todos os símbolos “úteis” já foram desempilhados — ao ver Z_0 no topo, sabe-se que a pilha está “logicamente vazia”; (ii) viabiliza a aceitação por pilha vazia de forma controlada: um λ -movimento $(q_1, \lambda, Z_0) \rightarrow (q_1, \epsilon)$ remove Z_0 , esvaziando de fato a pilha somente no momento correto (entrada consumida e contagem casada). Sem Z_0 , não haveria como distinguir “pilha vazia por sucesso” de “pilha que esvaziou cedo demais”, nem garantir que $n \geq 1$.

Parte 3 — Máquina de Turing

1. Por que $L_4 = a^n b^n c^n$ não é reconhecida por um AP? Qual propriedade da MT permite? L_4 não é livre de

contexto, logo nenhum AP a reconhece — o que se prova pelo Lema do Bombeamento para linguagens livres de contexto. Tomando $z = a^p b^p c^p$ e qualquer decomposição $z = uvwxy$ com $|vwx| \leq p$ e $|vx| \geq 1$, o trecho vwx abrange no máximo dois dos três blocos; bombeando, ao menos um símbolo deixa de crescer na mesma proporção, quebrando a igualdade das três contagens. A causa intuitiva é que a **pilha única** do AP, ao comparar a s com b s, é consumida e perde a informação necessária para depois comparar com os c s. A MT supera isso porque sua **fita é de leitura e escrita, com movimento bidirecional**: ela pode marcar símbolos e visitar qualquer posição quantas vezes for preciso, mantendo e reutilizando as três contagens simultaneamente.

2. Quantos estados a MT de L4 usou? Estratégia de marcação. A MT de L4 usa 8 estados: q_0 (busca o próximo a), q_1 (busca o próximo b), q_2 (busca o próximo c), q_3 (retorna ao início), q_4 e q_5 (verificação final) e os estados de parada q_{accept} e q_{reject} . A **estratégia de marcação** funciona em rodadas: em cada rodada marca-se um a como X , caminha-se à direita até o primeiro b marcando-o como Y , continua-se até o primeiro c marcando-o como Z , e então retorna-se à esquerda até o X mais recente para iniciar nova rodada. Quando q_0 encontra Y (em vez de a), não há mais a s; passa-se à verificação, que aceita apenas se restarem Y s seguidos de Z s e depois o branco — garantindo que as três quantidades são iguais.

3. Um computador moderno é mais poderoso que uma MT? (Tese de Church-Turing). Não. Segundo a Tese de Church-Turing, qualquer função efetivamente calculável pode ser computada por uma Máquina de Turing; nenhum modelo físico de computação conhecido ultrapassa esse limite de **computabilidade**. Um computador moderno é mais rápido e conveniente, mas é, na verdade, *menos* poderoso em um sentido teórico: possui memória **finita**, ao passo que a MT dispõe de fita infinita. Tudo o que um computador real calcula, uma MT também calcula (e há problemas, como o da parada, que nenhum dos dois resolve). A diferença é de eficiência e praticidade, não de poder computacional fundamental: ambos são limitados pela mesma fronteira da computabilidade.